```
"""
StegSim drift.py — Environmental drift and mutation engine.
Applies per-step drift to agents and environment.
This is what creates the governance collapse scenario.
v2+: topology mutation, adversarial pressure, reconstruction hooks.
"""

import random
import numpy as np
from .model import AgentState, SystemState, EnvironmentState, ViabilityMargin


class DriftEngine:
    """
    Applies drift to system state each step.
    Drift is deterministic given the RNG seed.
    """

    def __init__(self, cfg, rng: random.Random, np_rng: np.random.Generator):
        self.cfg    = cfg
        self.d      = cfg.drift
        self.rng    = rng
        self.np_rng = np_rng

    def apply(self, state: SystemState) -> SystemState:
        env    = state.environment
        agents = state.agents

        # ── Environment drift ──────────────────────────────────────

        # Resource scarcity increases
        env.resource_scarcity = min(0.99,
            env.resource_scarcity + self.d.resource_concentration_per_step
        )

        # Trust decay rate increases
        env.trust_decay_rate = min(0.05,
            env.trust_decay_rate + self.d.trust_decay_per_step * 0.1
        )

        # Policy version increments (global policy advances)
        if self.rng.random() < 0.1:  # policy updates ~10% of steps
            env.policy_version += 1
```

```
        # Policy lag grows as agents fail to sync
        env.policy_lag = min(1.0,
            env.policy_lag + self.d.policy_lag_growth_per_step
        )

        # Mutation pressure increases
        env.mutation_pressure = min(0.99,
            env.mutation_pressure + self.d.mutation_pressure_growth_per_step
        )

        # Fragmentation increases
        env.fragmentation = min(0.99,
            env.fragmentation + self.d.fragmentation_growth_per_step
        )

        # Communication fidelity degrades
        env.communication_fidelity = max(0.01,
            env.communication_fidelity - self.d.communication_decay_per_step
        )

        # External shock (episodic)
        if self.rng.random() < self.d.shock_probability:
            env.external_shock = self.d.shock_magnitude
            self._apply_shock(agents, env)
        else:
            env.external_shock = max(0.0, env.external_shock * 0.5)

        # ── Agent drift ──────────────────────────────────────────────

        # Sample a subset of agents for efficiency (all in small runs, 10% in lar
        n = len(agents)
        if n <= 1000:
            sample = list(agents.values())
        else:
            sample_size = max(100, n // 10)
            sample = self.rng.sample(list(agents.values()), sample_size)
            scale  = n / sample_size

        for agent in sample:
            self._drift_agent(agent, env)

        # ── Recompute viability ──────────────────────────────────────
        state.viability = self._compute_viability(state)

        return state

    def _drift_agent(self, agent: AgentState, env: EnvironmentState):
```

```
        """Apply per-step drift to one agent."""
        d = self.d

        # Trust decays
        agent.trust = max(0.01,
            agent.trust - env.trust_decay_rate * (1.0 + agent.mutation_pressure)
        )

        # Authority becomes stale (agent doesn't re-derive from current state)
        # This is the key: authority drifts from what current state would support
        agent.authority = max(0.01,
            agent.authority - d.authority_staleness_per_step
        )

        # Resources concentrate (Pareto drift)
        if self.rng.random() < 0.3:
            delta = self.np_rng.normal(0, 0.02)
            agent.resources = float(max(0.01, min(0.99, agent.resources + delta))

        # Mutation pressure accumulates
        agent.mutation_pressure = min(0.99,
            agent.mutation_pressure + d.mutation_pressure_growth_per_step
        )

        # Policy snapshot ages (agents don't always sync)
        # Sync probability decreases with communication degradation
        sync_prob = env.communication_fidelity * 0.3
        if self.rng.random() < sync_prob:
            agent.policy_snapshot_version = env.policy_version
        # Else: snapshot remains stale — this is the stale authority condition

        # Autonomy drift: agents tend to expand autonomy under pressure
        if agent.mutation_pressure > 0.2:
            agent.autonomy = min(0.99,
                agent.autonomy + d.mutation_pressure_growth_per_step * 0.5
            )

    def _apply_shock(self, agents: dict, env: EnvironmentState):
        """Apply external shock: sudden trust loss across a subset of agents."""
        n_affected = max(1, int(len(agents) * env.external_shock * 0.3))
        affected   = self.rng.sample(list(agents.values()), min(n_affected, len(a
        for agent in affected:
            agent.trust = max(0.01, agent.trust - env.external_shock * 0.5)
            agent.mutation_pressure = min(0.99,
                agent.mutation_pressure + env.external_shock * 0.3
            )
```

```
    def _compute_viability(self, state: SystemState) -> ViabilityMargin:
        """Recompute viability from current state after drift."""
        env = state.environment
        agents = state.agents

        if agents:
            mean_trust  = sum(a.trust for a in agents.values()) / len(agents)
            mean_auth   = sum(a.authority for a in agents.values()) / len(agents)
            gv          = env.policy_version
            mean_fresh  = sum(a.policy_freshness(gv) for a in agents.values()) /
        else:
            mean_trust = mean_auth = mean_fresh = 0.0

        components = {
            "trust_continuity":    float(max(0.0, min(1.0, mean_trust))),
            "authority_coherence": float(max(0.0, min(1.0, mean_auth))),
            "resource_balance":    float(max(0.0, min(1.0, 1.0 - env.resource_sca
            "policy_freshness":    float(max(0.0, min(1.0, mean_fresh))),
            "mutation_pressure":   float(max(0.0, min(1.0, env.mutation_pressure)
            "fragmentation":       float(max(0.0, min(1.0, env.fragmentation))),
        }

        weights = {
            "trust_continuity":    self.cfg.weights.trust_continuity,
            "authority_coherence": self.cfg.weights.authority_coherence,
            "resource_balance":    self.cfg.weights.resource_balance,
            "policy_freshness":    self.cfg.weights.policy_freshness,
            "mutation_pressure":   self.cfg.weights.mutation_pressure,
            "fragmentation":       self.cfg.weights.fragmentation,
        }

        return ViabilityMargin.compute(weights, components)


# ── v2+ Topology mutation stub ──────────────────────────────────────────

class TopologyMutationEngine:
    """
    v2: Dynamic topology mutation — trust fragmentation, authority concentration,
    communication collapse, isolated subgraphs.
    Not active in v1 (enable_topology_mutation=False).
    """

    def apply(self, state: SystemState, rng: random.Random) -> SystemState:
        raise NotImplementedError("Topology mutation is a v2 feature.")
```